Audit Report

Own Protocol v2

Date: **October 18 — November 1, 2026**

Introduction

eUSD is an overcollateralized dollar stablecoin issued against tokenized real-world assets (RWAs). Holders of tokenized equities and other on-chain collateral, such as Robinhood stock tokens and WETH, lock them in an `EUSDManager` and mint eUSD against a fresh oracle price, keeping their exposure to the underlying asset. Each collateral risk group runs its own manager with its own ratios, fee and debt ceiling, and every manager mints the same eUSD. A fixed annual stability fee accrues on principal and is minted to the protocol treasury; keepers liquidate unsafe positions at a fixed bonus; and Liquity-style redemption against the riskiest positions anchors the peg at $1.

Collateral is priced by `ChainlinkOracleVerifier`: Chainlink feeds are the primary source, and band-limited, EIP-712-signed in-house quotes fill in only while a feed is quiet, so tokenized stocks can still be valued when their home market is closed. `StakingVault` is a fixed-rate ERC-4626 vault for staking eUSD (seUSD) and $MONEY (sMONEY), funded from the treasury. `ProtocolRegistry` holds roles and shared addresses, and `MoneyFeeCollector` collects protocol fees and buys back $MONEY.

This is the pre-deployment review of the eUSD contract set. Three independent multi-agent adversarial audits were run on the audited commit, each with nine specialist attack agents and three gap-hunting agents. Their results were merged, deduplicated against the earlier report, and every item was re-verified against source before it was booked. After the fixes, a final full pass ran on the re-audit commit; its new items are included below (L-06 to L-08). Every fix carries a regression test that fails without it.

Scope

Repository: `own-protocol/own-v2`

Audited Commit: `c5d94b1c1ba02454382556f184adde4c6090a6ba`

Re-audit Commit: `67bff7178efc342d970fec8c6ba86caac8ca01ad`

Fixes Through: `329f8f68c1277ac8059858732a3fbbdfde3250be`

Files:

- `src/eusd/EUSDManager.sol`
- `src/eusd/EUSD.sol`
- `src/oracle/ChainlinkOracleVerifier.sol`
- `src/staking/StakingVault.sol`
- `src/registry/ProtocolRegistry.sol`
- `misc/MoneyFeeCollector.sol`

Interfaces were reviewed with their contracts. `EUSD` is already deployed and non-upgradeable; it was re-read for completeness and is unchanged by this release.

Findings

Each issue has an assigned severity:

Critical/High issues are directly exploitable security vulnerabilities that can lead to loss of funds and need to be fixed.

Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.

Low/Info issues are non-exploitable or informational findings that do not pose a direct security risk to the system's integrity.

Issues marked `RESOLVED` are fixed in the codebase and carry a regression test that fails without the fix. Issues marked `ACKNOWLEDGED` are accepted by design, with the rationale and any operational mitigations documented.

Issues Found

Critical	High	Medium	Low	Info
0	0	1	8	3

Issues Not Fixed and Not Acknowledged

Critical	High	Medium	Low	Info
0	0	0	0	0

Summary of Findings

ID	Severity	Title	Status
M-01	Medium	`redeem` priced collateral at an unbounded-age oracle price with no redemption fee	Fixed
L-01	Low	Permissionless `accrue` compounded the stability fee	Fixed
L-02	Low	`closePositionPrincipalOnly` settled the fee at an unbounded-age price	Fixed
L-03	Low	In-house price cache survives reconfiguration and signer removal	Acknowledged
L-04	Low	Full liquidation pushes the refund to the owner inline	Acknowledged
L-05	Low	One OPERATOR role covers the oracle kill-switch, mint pause and reward top-ups	Acknowledged

L-06	Low	Off-hours redemption can use an in-house price up to `mintPriceMaxAge` old	Acknowledged
L-07	Low	`StakingVault` rewards compound on every touch, above `aprBps`	Acknowledged
L-08	Low	Repayments retired principal before fees, shrinking the fee base	Fixed
I-01	Info	Principal-only close and `pullFromTreasury` draw on the same treasury eUSD	Acknowledged
I-02	Info	`setChainlinkConfig` does not bound `bandBps` to `BPS`	Acknowledged
I-03	Info	`buyBack` forwards the full held $MONEY, not just the swap output	Acknowledged

Findings that restated an earlier, already-decided issue were not booked again; their current status is in the next section.

Status of Earlier Findings

IDs below refer to the eUSD and oracle report for August 26 — September 9, 2026. Every finding of that report was re-checked against the current code; only those whose status or code changed since are listed. The rest stand as recorded there.

- **L-03** (no `minDebt` floor on the redemption path): fixed after the September review. A partial redemption now leaves a position either fully closed or at or above `minDebt`.
- **H-02**, **M-03**, **L-08** (stock splits, halted collateral, eToken dividends): the code paths no longer exist. Collateral is now the issuer's token itself, priced by Chainlink feeds that already include the token multiplier; halt handling is replaced by the mint pause and per-collateral disable.
- **L-04** (stale-anchor exits): redemption now requires a fresh price (M-01 above). Liquidation keeps the September decision.
- **I-03** (zero-fee redemption): reconsidered under M-01; still no fee, by decision.
- **I-11** (`setChainlinkConfig` keeps the old in-house price cache): re-raised in wider form as L-03 above and acknowledged with operating rules.
- **I-10** (`_verifyInhouseProof` staleness): the in-proof verification path was removed; in-house prices enter only through `updatePrice`, which enforces `inhouseMaxStaleness`.
- **L-11** (a reverting aggregator bricks both price legs): unchanged and still acknowledged. No configured ticker sets a multiplier token.

Properties Attacked and Held

- **EUSDManager:** eUSD supply equals the sum of `totalDebt` across mint, repay, every `_close` branch, liquidation and redemption; `totalCollateral` matches token flows; a position is listed if and only if it has both debt and collateral; partial liquidation and redemption never lower the survivor's ratio; rounding favours the protocol; UUPS initializers are locked.
- **ChainlinkOracleVerifier:** the EIP-712 domain is bound to the verifier address (no replay from an earlier verifier), future timestamps and zero prices are rejected, `clDecimals <= 18`.
- **StakingVault:** no share-price inflation (donations ignored, `_staked` tracked), no back-pay after an empty reserve or zero supply, `setApr` / `setCap` accrue at the old rate first, payouts bounded by stake plus reserve.
- **ProtocolRegistry:** two-step, delayed admin transfer.
- **MoneyFeeCollector:** append-only storage across the upgrade, allowance reset after swaps, balance-delta claim accounting.

No unprivileged path to take user or protocol funds survived verification.

Issue M-01: `redeem` priced collateral at an unbounded-age oracle price with no redemption fee RESOLVED

Severity: Medium

Vulnerability Detail

`redeem` read `(price,) = _oracle().getPrice(ticker)` and discarded the timestamp, while `mint` and `withdrawCollateral` go through `_freshPrice`. The verifier serves a Chainlink answer up to `maxAnchorAge` old, and an in-house quote may only sit within `bandBps` of that anchor (`_checkAnchorBand`), so even an honest, always-on signer cannot follow an off-hours move larger than the band. Tokenized stocks keep trading on-chain while their feeds are silent, and there is no redemption fee.

When the token trades above the stale anchor by more than the band, a redeemer buys eUSD at or below $1, redeems against the head positions at the stale price and sells the collateral at market. Worked case: anchor $600, in-house capped at $618 by a 3% band, token trading at $650 over a weekend gap. Redeeming 100,000 eUSD seizes 161.8 tokens worth $105,178, a ~$5,000 transfer from head debtors, who cannot defend off-hours (mint and withdraw need a fresh price; only `closePosition` with the full debt in eUSD is open). With several managers the redeemer picks the most-lagged collateral.

The September review acknowledged the stale-anchor exit family against the liquidation bonus (L-04 of that report). The redemption-side transfer from head debtors, its dependence on `bandBps`, and the missing fee were not covered by that decision, so the issue is booked here.

Impact

Value transfer from the riskiest (head) borrowers to redeemers whenever collateral trades away from a stale anchor, repeatable for as long as the gap persists.

Resolution

`redeem` now prices through `_freshPrice` (no older than `mintPriceMaxAge`, the same bound as mint), so collateral is never redeemed at a stale price; with no fresh price

the call reverts `StaleMintPrice` and waits. Off-hours redemption follows the in-house signer, whose anchor band is an admin setting sized so it can track off-hours moves; a move beyond the band pauses redemption until prices catch up.

A redemption fee was considered and not added: it only deters gaps smaller than the fee and loosens the peg floor by the same amount. The residual in-session gap (the Chainlink deviation threshold) is accepted; the manager is upgradeable if a fee is ever needed. Liquidation keeps the September decision. Test: `test_redeem_stalePrice_reverts_thenFreshPriceWorks`; the invariant handler re-posts the price.

Issue L-01: Permissionless `accrue` compounded the stability fee RESOLVED

Severity: Low

Vulnerability Detail

`_accrue` charged `fee = p.debt × Δindex / (BPS · YEAR)` and folded it into `p.debt`, so the next fee was charged on principal plus earlier fees. `accrue(collateral, owner, hint)` is callable by anyone on any position. Untouched for a year, 1,000,000 debt at 8% owes 80,000; touched daily by a third party it owed 83,277; hourly, 83,287 (the e^r − 1 limit).

Impact

Borrower griefing at gas cost only: up to ~0.33% of debt per year extra, minted to the treasury, so no attacker profit. It contradicted the interface ("simple interest", and `accrue` "never changes what a position owes").

Resolution

`_accrue` and `currentDebt` charge the fee on principal only (`p.debt - p.feesAccrued`), so the fee is simple interest however often a position is touched. Debt that is only unpaid fees accrues no further fee. Test: `test_fee_frequentAccrue_doesNotCompound` (365 daily third-party touches match an untouched position to within rounding dust).

Issue L-02: `closePositionPrincipalOnly` settled the fee at an unbounded-age price RESOLVED

Severity: Low

Vulnerability Detail

`_close(…, feesFromCollateral = true)` priced the fee collateral with raw `getPrice` and burned the full fee from the treasury. The debtor picks `closePosition` (fee in eUSD, no price) or `closePositionPrincipalOnly` (fee in collateral at the oracle price) after seeing the price, so the treasury was always on the losing side of any staleness. Worked case: fees 80 eUSD, stale price $600, market $540; the treasury retires 80 eUSD for $72 of collateral.

Impact

Bounded by `feesAccrued × price gap` per position; contradicted the documented "value-neutral for the treasury" property.

Resolution

The `feesFromCollateral` branch prices through `_freshPrice`. `closePosition` stays the price-free exit that always works off-hours, and a principal-only close with no fees to settle still needs no price. Test: `test_closePrincipalOnly_stalePrice_reverts_closePositionStillWorks`.

Issue L-03: In-house price cache survives reconfiguration and signer removal; replay after re-enable ACKNOWLEDGED

Severity: Low

Vulnerability Detail

Only `disableAsset` deletes `_prices[asset]`. After `setChainlinkConfig` re-points a ticker (new aggregator or `multiplierToken`), `getPrice` keeps serving the last in-house price, in the old units and never re-band-checked, while it is newer than the Chainlink answer and within `inhouseMaxStaleness`; the same holds for a price signed by a signer that `removeSigner` just revoked. Conversely, because `disableAsset` zeroes the stored timestamp, the replay floor (`timestamp <= existing.timestamp`) is lost: after a disable and re-configuration, anyone can resubmit an earlier public attestation that is still within staleness and band.

Impact

Every path needs an admin sequence and is bounded by `inhouseMaxStaleness` and the anchor band, but liquidation reads the price with no age bound.

Resolution

Acknowledged and handled operationally; the verifier is unchanged. Replay only accepts signatures from signers that are still authorized and prices within `inhouseMaxStaleness`, and any newer attestation overwrites the cache and raises the replay floor. Operating rules: push a fresh price right after every `setChainlinkConfig` or re-enable; on a signer compromise run `removeSigner(old)`, `addSigner(new)` and push fresh prices for every ticker.

Issue L-04: Full liquidation pushes the refund to the owner inline (a blocked owner reverts it) ACKNOWLEDGED

Severity: Low

Vulnerability Detail

On a full close, `liquidate` sends `refund` to the owner in the liquidator's call. Robinhood stock tokens gate `transfer` with `onlyNotBlocked(to)` (verified on-chain), so if the token issuer blocks the owner, every full liquidation with `refund > 0` reverts. Partial liquidations still work but must leave at least `minDebt`.

Impact

The last `minDebt` of a blocked owner's position cannot be cleared by liquidation (redemption can still reach it). Capped at about `minDebt` of debt per blocked position.

Resolution

Acknowledged: issuer blocklisting is rare, the impact is capped and still reachable by redemption, and a pull-pattern refund would add a claim step for every liquidated owner. Revisit by upgrade if a blocked owner ever holds a position.

Issue L-05: One OPERATOR role covers the oracle kill-switch, mint pause and reward top-ups ACKNOWLEDGED

Severity: **Low**

Vulnerability Detail

`disableAsset` / `removeSigner` (verifier), `setMintPaused` (managers) and `pullFromTreasury` (vaults) share the registry OPERATOR role, and it is granted to the keeper so it can top up the vaults.

Impact

A compromised keeper could `disableAsset` (every `getPrice` reverts, freezing liquidation, redemption and principal-only close until ADMIN re-configures) or pause mints. It cannot move funds anywhere but treasury → vault reserve, capped by the treasury's allowance.

Resolution

By design. The keeper key is held in a cloud KMS; the realistic compromise is the server, which would expose a separate rewards key too, so a dedicated role adds little. Containment: ADMIN re-configures, and `withdrawReserve` returns over-pulled rewards.

Issue L-06: Off-hours redemption can use an in-house price up to `mintPriceMaxAge` old ACKNOWLEDGED

Severity: Low

Vulnerability Detail

After the M-01 fix, `redeem` accepts any price no older than `mintPriceMaxAge`. Off-hours the stored price is the last in-house push, so a redeemer can redeem against it after the market has moved. `updatePrice` is permissionless and accepts any valid signature newer than the stored one (never an older one), and the oracle service publishes each signed price, so a caller can also push a signed price from between two keeper pushes and redeem in the same transaction.

Impact

The transfer from head debtors is capped by the token's price move inside one push interval, and by the anchor band.

Resolution

Acknowledged. The anchor band is set on every ticker at go-live, and large off-hours moves within one push interval are rare for the listed collateral. If they become material, the keeper pushes more often; no contract change. Same class as the in-session residual accepted under M-01.

Issue L-07: `StakingVault` rewards compound on every touch, above `aprBps` ACKNOWLEDGED

Severity: Low

Vulnerability Detail

`_accrue` adds `earned` to `_staked`, and `_pending` charges `aprBps × min(_staked, cap)`, so below the cap rewards earn rewards. `deposit(0)`, `fund(1)` and every deposit or withdraw are touches, so the effective rate trends to e^apr − 1 (10% APR touched every block pays about 10.52%). Above the cap the drip is fixed and nothing compounds.

Impact

The admin-funded reserve pays the difference; no user loses funds.

Resolution

Acknowledged: the extra is a few percent of the reward budget. The configured rate is treated as an APY when sizing the reserve.

Issue L-08: Repayments retired principal before fees, shrinking the fee base RESOLVED

Severity: Low

Vulnerability Detail

A side effect of the L-01 fix. The fee is charged on `debt - feesAccrued`, but `repay`, `liquidate` and `_redeemFrom` only clamped `feesAccrued` down to the new debt, so a repayment reduced principal first. Worked case at 8%: 1,000 debt grows to 1,080 (80 fee); the borrower repays 80 and still owes 1,000, but year 2 charges 8% on 920 (73.6, not 80).

Impact

The treasury lost 8% a year of every fee a borrower had already repaid while the position stayed open.

Resolution

Every debt reduction retires fees first (`p.feesAccrued -= Math.min(repaid, p.feesAccrued)`) in `repay`, `liquidate` and `_redeemFrom`; `feesAccrued <= debt` still holds. Test: `test_repay_retiresFeesFirst`.

Issue I-01: Principal-only close and `pullFromTreasury` draw on the same treasury eUSD ACKNOWLEDGED

Severity: **Info**

Vulnerability Detail

`closePositionPrincipalOnly` burns the settled fee from `registry.treasury()`, and `StakingVault.pullFromTreasury` pulls eUSD from the same address.

Impact

If top-ups take the treasury's eUSD below outstanding `feesAccrued`, principal-only closes revert (documented; `closePosition` still works).

Resolution

Acknowledged as an operating rule: keep the treasury's eUSD at or above the sum of open `feesAccrued`, or fund the seUSD vault from other eUSD.

Issue I-02: `setChainlinkConfig` does not bound `bandBps` to BPS ACKNOWLEDGED

Severity: Info

Vulnerability Detail

`bandBps` is a `uint16` up to 65,535; at 10,000 or more any lower in-house price passes `_checkAnchorBand`.

Impact

Needs an admin typo plus a signer compromise.

Resolution

Acknowledged: the band is set in the reviewed go-live Safe batch; the verifier stays unchanged.

Issue I-03: `buyBack` forwards the full held $MONEY, not just the swap output ACKNOWLEDGED

Severity: Info

Vulnerability Detail

For a non-$MONEY input, `buyBack` forwards `money_.balanceOf(this)`, so pre-existing $MONEY moves to `reserve` in one call regardless of `burnSpendBps`. Separately, a compromised keeper can route the per-interval input slice to itself through permissive `swapData` with `minMoneyOut = 1`; the contract documents that slice as the keeper's damage cap.

Impact

The destination is the owner-set reserve (the treasury Safe), so the first behaviour is a sweep of donations and swap dust, not a bypass of the keeper cap on the input token.

Resolution

By design. Keep `burnInterval` and `burnSpendBps` conservative.